

SAT encodings for empty-board N-Queens: a comparison with ILP and CP

Vu Minh Hoang^{a,*}

^aVNU University of Engineering and Technology, Hanoi, Vietnam

ARTICLE INFO

Keywords:

N-Queens Problem
SAT Encoding
At-Most-One Constraint
Combinatorial Optimization
Integer Linear Programming
Constraint Programming
CPLEX
Gurobi
CP-SAT

ABSTRACT

The choice of at-most-one (AMO) encoding changes the size and propagation behavior of a Boolean satisfiability (SAT) formula for N-Queens. An archived benchmark has unequal method coverage, so it cannot compare all five encodings at large sizes. This study implements pairwise, binary, commander, sequential counter, and product encodings for empty-board N-Queens and compares them with integer linear programming and constraint programming models. Two clean-source runs test the five SAT encodings, CBC, Gurobi, and CP-SAT at $N \in \{20, 50, 100, 150, 200\}$, with three repetitions and a 120-s full-process deadline per case. At $N = 200$, median construction-plus-solver-call time is 2.252 s for Commander/Glucose4, 0.421 s for Gurobi, and 44.776 s for CP-SAT. Sequential and CBC time out in all three repetitions. Exact formula counts and unit-propagation tests check the implemented encodings. The results describe one machine and five feasible board sizes; they do not establish general solver superiority.

1. Introduction

The N-Queens problem asks for N non-attacking queens on an $N \times N$ board. Direct constructions find a placement for $N = 1$ and every $N \geq 4$ in polynomial time [3]. The empty-board problem still permits a controlled comparison of SAT, integer linear programming (ILP), and constraint programming (CP) formulations. Its results do not extend automatically to N-Queens Completion or unrelated combinatorial problems [6].

SAT solvers use conflict-driven clause learning (CDCL), but they require a conjunctive normal form (CNF) representation of the placement constraints [12]. Different at-most-one (AMO) encodings can change both CNF size and what unit propagation deduces [17]. The archived benchmark includes only the binary encoding at $N = 150$ and $N = 200$. It therefore cannot compare all five SAT encodings at those sizes.

This study asks how the five encodings differ in formula size and measured runtime under a common process limit. It also compares SAT with ILP and CP-SAT on the same empty-board sizes. The paper checks exact CNF counts and AMO propagation, then reports archived and clean-source measurements separately. The study does not propose a new encoding or claim a general solver ranking.

2. Background and related work

2.1. The N-Queens problem

A solution places one queen in each row and column and at most one on each diagonal. Empty boards with $N = 2$ or $N = 3$ have no solution. N-Queens Completion adds pre-placed queens; its decision and counting versions are NP-complete and #P-complete, respectively [6]. This study seeks one placement on an empty board.

2.2. Boolean satisfiability (SAT)

SAT asks whether a Boolean formula has a satisfying assignment. In CNF, the formula is a conjunction of clauses, and each clause is a disjunction of literals.

Modern SAT solvers extend the Davis-Putnam-Logemann-Loveland (DPLL) procedure with CDCL [12, 13, 15]. Glucose uses literal block distance (LBD), the number of decision levels in a learned clause, to assess learned clauses [1, 2]. The SAT experiments use Glucose4 through PySAT.

*Corresponding author

 24021494@vnu.edu.vn (V.M. Hoang)

2.3. Cardinality constraints

Cardinality constraints limit the number of true variables in $X = \{x_1, x_2, \dots, x_n\}$. The N-Queens model uses three forms:

- **At least one (ALO):** $\sum x_i \geq 1$, encoded by the clause $(x_1 \vee x_2 \vee \dots \vee x_n)$.
- **At most one (AMO):** $\sum x_i \leq 1$. Several CNF encodings trade formula size against propagation strength [5].
- **Exactly one (EXO):** $\sum x_i = 1$, equivalent to $\text{ALO} \wedge \text{AMO}$.

2.4. Integer linear programming (ILP)

ILP uses linear constraints and integer-valued decision variables [16]. Solvers can combine linear relaxations with branch-and-bound and cutting planes. The baselines use Gurobi [8], IBM ILOG CPLEX [9], and CBC through PuLP [14].

2.5. Constraint programming (CP)

CP searches for assignments that satisfy stated constraints. Global constraints such as `AllDifferent` can use specialized filtering, including Régin's matching-based method [18, 19]. The baselines use OR-Tools CP-SAT [7] and IBM ILOG CPLEX CP Optimizer [9]. Their internal propagation cannot be determined from the model alone.

2.6. Related work

Prestwich reviewed CNF translations of constraint satisfaction problems [17]. Pairwise AMO uses $O(n^2)$ clauses; Sinz's sequential counter reduces this to $O(n)$ clauses and auxiliary variables [20]. Klieber and Kwon proposed hierarchical commander groups [11], while Chen studied grid and product encodings [4]. All five encodings in this paper are established methods.

3. SAT encodings for N-Queens

3.1. Problem formulation

For each cell (i, j) of an $N \times N$ board, let $x_{i,j}$ be true exactly when that cell contains a queen. Here $i, j \in \{1, \dots, N\}$.

The board-level constraint has six components. For each encoding E , auxiliary variables Y are allocated separately for each AMO set, and the resulting CNF $\Phi_E(X, Y)$ satisfies $\Phi_{\text{board}}(X) \iff \exists Y \Phi_E(X, Y)$:

$$\Phi = \Phi_{\text{ALO}}^{\text{row}} \wedge \Phi_{\text{AMO}}^{\text{row}} \wedge \Phi_{\text{ALO}}^{\text{col}} \wedge \Phi_{\text{AMO}}^{\text{col}} \wedge \Phi_{\text{AMO}}^{\text{diag}} \wedge \Phi_{\text{AMO}}^{\text{anti}} \quad (1)$$

3.2. Row and column constraints

Every row and column must contain exactly one queen. For row i , the ALO clause is

$$\bigvee_{j=1}^N x_{i,j} \quad (2)$$

The chosen encoding supplies row AMO clauses; columns use the same construction. Each row and column therefore satisfies $\text{EXO} = \text{ALO} \wedge \text{AMO}$.

3.3. Diagonal constraints

Cells share a main diagonal when $d = i - j$ is equal, with $d \in \{-(N-1), \dots, N-1\}$. They share an anti-diagonal when $s = i + j$ is equal, with $s \in \{2, \dots, 2N\}$. Each diagonal needs AMO but may be empty, so it needs no ALO clause.

3.4. AMO encoding schemes

For n Boolean variables $X = \{x_1, \dots, x_n\}$, AMO permits at most one true input:

$$\sum_{i=1}^n x_i \leq 1 \quad (3)$$

The five implementations encode this constraint in CNF with different auxiliary-variable and clause counts. All five propagate a true input to the falsity of every other AMO input.

3.4.1. Binomial (pairwise) encoding

The pairwise encoding forbids every pair of inputs from being true together:

$$\Phi_{\text{binomial}} = \bigwedge_{1 \leq i < j \leq n} (\neg x_i \vee \neg x_j) \quad (4)$$

It uses no auxiliary variables and $\binom{n}{2} = O(n^2)$ binary clauses per AMO constraint. Across the board, the clause count is $O(N^3)$. If x_i is true, unit propagation falsifies every other input x_j . This is generalized arc consistency (GAC) on the original AMO inputs.

3.4.2. Binary encoding

Binary encoding assigns each input a code over $m = \lceil \log_2 n \rceil$ auxiliary bits $Y = \{y_0, \dots, y_{m-1}\}$ [5]. Each index $k \in \{0, \dots, n-1\}$ has a unique expansion $k = \sum_{b=0}^{m-1} \text{bit}(k, b) 2^b$.

The literal required by bit b of code k is

$$\ell(k, b) = \begin{cases} y_b & \text{if bit } b \text{ of } k \text{ is 1,} \\ \neg y_b & \text{if bit } b \text{ of } k \text{ is 0.} \end{cases} \quad (5)$$

The binary AMO formula is

$$\Phi_{\text{binary}} = \bigwedge_{k=0}^{n-1} \bigwedge_{b=0}^{m-1} (\neg x_k \vee \ell(k, b)) \quad (6)$$

If x_k is true, unit propagation fixes every bit in Y . Each other input requires a different bit in at least one position, so its corresponding clause propagates $\neg x_j$. This establishes GAC on the original AMO inputs, not propagation completeness for arbitrary auxiliary assignments. The encoding uses $\lceil \log_2 n \rceil$ auxiliary variables and $n \lceil \log_2 n \rceil$ binary clauses per constraint.

3.4.3. Commander encoding

Commander encoding partitions X into groups $G_1, \dots, G_m$ of at most three inputs and applies the construction recursively [11]. Each group has one auxiliary commander variable c_k .

The encoding has four clause sets:

1. **Within-group AMO** (Φ_{intra}): Forbid two true inputs in one group:

$$\Phi_{\text{intra}} = \bigwedge_{k=1}^m \bigwedge_{\substack{x_i, x_j \in G_k \\ i < j}} (\neg x_i \vee \neg x_j) \quad (7)$$

2. **Forward implication** (Φ_{fwd}): A true group input forces its commander true:

$$\Phi_{\text{fwd}} = \bigwedge_{k=1}^m \bigwedge_{x \in G_k} (\neg x \vee c_k) \quad (8)$$

3. **Reverse implication** (Φ_{rev}): A true commander requires a true group input:

$$\Phi_{\text{rev}} = \bigwedge_{k=1}^m \left(\neg c_k \vee \bigvee_{x \in G_k} x \right) \quad (9)$$

4. **Commander AMO** (Φ_{cmd}): Apply AMO recursively to the commanders:

$$\Phi_{\text{cmd}} = \text{AMO}(\{c_1, c_2, \dots, c_m\}) \quad (10)$$

The recursion terminates at $n \leq 3$ with pairwise AMO. The implementation includes reverse implications, so $c_k \leftrightarrow \bigvee_{x \in G_k} x$. Reverse clauses are optional for the projected AMO relation and input GAC. Forward implications, within-group AMO, and recursive commander AMO already propagate $\neg x_j$ after x_i becomes true [11]. The implemented version uses $O(n)$ auxiliary variables and clauses.

3.4.4. Sequential counter encoding

Sinz's sequential counter uses $n - 1$ auxiliary variables $s_1, \dots, s_{n-1}$ [20]. They carry the prefix implications needed to reject a second true input. The clauses do not force each s_i to equal the prefix disjunction in every satisfying auxiliary assignment.

The clauses enforce prefix propagation and collision prohibition:

$$(\neg x_1 \vee s_1) \quad (11)$$

$$\bigwedge_{i=2}^{n-1} ((\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1})) \quad (12)$$

$$(\neg x_n \vee \neg s_{n-1}) \quad (13)$$

The encoding uses $n - 1 = O(n)$ auxiliary variables and $3n - 4 = O(n)$ clauses per constraint. If x_i is true, unit propagation follows the prefix chain to falsify later inputs and propagates backward to falsify earlier inputs. It is GAC on the original AMO inputs.

3.4.5. Product encoding

Product encoding places the n inputs on a two-dimensional grid [4]. It sets $p = \lceil \sqrt{n} \rceil$ and $q = \lceil n/p \rceil$, so $n \leq pq$. The auxiliary coordinates are $U = \{u_1, \dots, u_p\}$ and $V = \{v_1, \dots, v_q\}$.

Each variable x_k ($k \in \{0, \dots, n - 1\}$) is mapped to grid coordinates $r = \lfloor k/q \rfloor + 1$ and $c = (k \bmod q) + 1$ via channeling clauses:

$$\Phi_{\text{chan}} = \bigwedge_{k=0}^{n-1} ((\neg x_k \vee u_{\lfloor k/q \rfloor + 1}) \wedge (\neg x_k \vee v_{(k \bmod q) + 1})) \quad (14)$$

Pairwise AMO constraints are then enforced independently across U and V :

$$\Phi_{\text{grid}} = \bigwedge_{1 \leq i < j \leq p} (\neg u_i \vee \neg u_j) \wedge \bigwedge_{1 \leq i < j \leq q} (\neg v_i \vee \neg v_j) \quad (15)$$

To eliminate invalid phantom cells when $p \times q > n$, phantom clauses forbid vacant coordinate pairs:

$$\Phi_{\text{phantom}} = \bigwedge_{r=1}^p \bigwedge_{\substack{c=1 \\ (r-1)q + (c-1) \geq n}}^q (\neg u_r \vee \neg v_c) \quad (16)$$

For $n < 6$, the implementation uses pairwise AMO instead of creating grid variables. For $n \geq 6$, a true x_k fixes its two coordinates. Coordinate AMO then falsifies every other input through at least one differing coordinate. The encoding is GAC on the original AMO inputs and uses $O(\sqrt{n})$ auxiliary variables and $O(n)$ clauses per constraint.

3.5. Baseline exact formulations

3.5.1. Constraint programming formulation (CP)

The CP model uses N integer variables $Q = \{q_1, \dots, q_N\}$. Variable $q_i \in \{1, \dots, N\}$ gives the queen's column in row i , so the variable indices enforce row occupancy. Three AllDifferent constraints enforce distinct columns and diagonals:

$$\text{AllDifferent}(q_1, q_2, \dots, q_N) \quad (17)$$

$$\text{AllDifferent}(q_1 + 1, q_2 + 2, \dots, q_N + N) \quad (18)$$

$$\text{AllDifferent}(q_1 - 1, q_2 - 2, \dots, q_N - N) \quad (19)$$

Régis's matching algorithm is one possible way to enforce GAC for AllDifferent [18]. This model does not establish which propagators or presolve steps either CP solver used in a particular run.

Table 1

Asymptotic size of the N-Queens SAT encodings

Encoding	Total variables	Total clauses	Input GAC via UP
Binomial (Pairwise)	N^2	$O(N^3)$	Yes
Binary	$N^2 + O(N \log N)$	$O(N^2 \log N)$	Yes
Commander	$N^2 + O(N^2)$	$O(N^2)$	Yes
Sequential Counter	$N^2 + O(N^2)$	$O(N^2)$	Yes
Product	$N^2 + O(N\sqrt{N})$	$O(N^2)$	Yes

3.5.2. Integer linear programming formulation (ILP)

The ILP model uses N^2 binary variables $x_{i,j}$, where $x_{i,j} = 1$ exactly when cell (i, j) contains a queen. Four families of linear constraints enforce non-attacking placement:

$$\sum_{j=1}^N x_{i,j} = 1 \quad \forall i \in \{1, \dots, N\} \quad (\text{Row constraints}) \quad (20)$$

$$\sum_{i=1}^N x_{i,j} = 1 \quad \forall j \in \{1, \dots, N\} \quad (\text{Column constraints}) \quad (21)$$

$$\sum_{\substack{1 \leq i, j \leq N \\ i-j=d}} x_{i,j} \leq 1 \quad \forall d \in \{-(N-1), \dots, N-1\} \quad (\text{Main diagonals}) \quad (22)$$

$$\sum_{\substack{1 \leq i, j \leq N \\ i+j=s}} x_{i,j} \leq 1 \quad \forall s \in \{2, \dots, 2N\} \quad (\text{Anti-diagonals}) \quad (23)$$

3.6. Complexity comparison

Table 1 gives asymptotic counts for the complete SAT models. An $N \times N$ board has N rows, N columns, and $O(N)$ diagonals, each with at most N cells.

Here, input GAC means that setting one input of an individual AMO constraint to true makes unit propagation falsify its other inputs. It does not imply complete propagation for the full N-Queens formula. For $N \geq 2$, let $a_E(k)$ and $c_E(k)$ count auxiliary variables and clauses for one k -input AMO, with both zero for $k \leq 1$. Exact board-level counts are

$$V_E(N) = N^2 + (2N + 2)a_E(N) + 4 \sum_{k=2}^{N-1} a_E(k), \quad (24)$$

$$C_E(N) = 2N + (2N + 2)c_E(N) + 4 \sum_{k=2}^{N-1} c_E(k). \quad (25)$$

The $2N$ term counts row and column ALO clauses. The one-constraint counts for pairwise, Binary, and Sequential are

$$(a_{\text{pair}}, c_{\text{pair}}) = (0, \binom{k}{2}), \quad (26)$$

$$(a_{\text{binary}}, c_{\text{binary}}) = (\lceil \log_2 k \rceil, k \lceil \log_2 k \rceil), \quad (27)$$

$$(a_{\text{seq}}, c_{\text{seq}}) = (k - 1, 3k - 4). \quad (28)$$

Product uses pairwise AMO for $k < 6$. For larger k , let $p = \lceil \sqrt{k} \rceil$ and $q = \lceil k/p \rceil$. Its counts are

$$(a, c) = \left(p + q, 2k + \binom{p}{2} + \binom{q}{2} + pq - k \right). \quad (29)$$

For Commander, groups have sizes $t_j \leq 3$ and $m = \lceil k/3 \rceil$. When $k > 3$, the recursive counts are

$$a(k) = m + a(m), \quad c(k) = \sum_j \binom{t_j}{2} + k + m + c(m). \quad (30)$$

Table 2

Variables and clauses generated by each SAT encoding

Encoding	$N = 20$		$N = 50$		$N = 100$	
	Vars	Clauses	Vars	Clauses	Vars	Clauses
Binomial	400	12,580	2,500	203,450	10,000	1,646,900
Binary	866	7,296	4,036	57,244	13,678	269,024
Commander	1,172	5,050	7,514	33,740	30,536	138,628
Sequential	1,882	4,372	12,202	28,912	49,402	117,812
Product	1,190	4,524	5,890	30,330	19,428	117,736

The pairwise case is the recursion base. The m reverse clauses are included in the count. These expressions count generated clauses before solver preprocessing.

4. Experimental results and comparison

4.1. Experimental setup

The archived dataset contains 116 aggregate rows across $N \in \{4, 8, 10, 15, 20, 25, 30, 40, 50, 75, 100, 150, 200\}$. The original description identifies a Windows machine with an Intel Core processor and 16 GB of RAM. It omits the exact CPU model, package versions, random seeds, solver thread counts, and individual runs. These gaps limit reproduction and statistical inference. The SAT models use Glucose4 through PySAT [10]. The baselines use CBC through PuLP, Gurobi, CPLEX MIP, OR-Tools CP-SAT, and CPLEX CP Optimizer [7–9, 14].

The archived CSV reports construction time, solver invocation time, and their sum in seconds. Solution extraction and equivalent postprocessing occur after the solver timer stops, so these totals exclude end-to-end latency. The historical scripts contain both a 120-s, two-repeat configuration and a 300-s, three-repeat configuration. Raw repetitions are unavailable, so the originating protocol of each archived row cannot be recovered. The revised runner records each status, deadline, wall time, solution validation, package version, and worker count. It leaves the archive unchanged and stores new measurements separately.

Solver coverage differs. Only Binary among the five SAT encodings was recorded at $N = 150$ and $N = 200$; Product ends at $N = 50$, while pairwise, Commander, and Sequential end at $N = 100$. CBC ends at $N = 100$. Gurobi and CP-SAT have recorded results through $N = 200$. Missing entries must be read as *not evaluated*, except entries marked with a license symbol. CPLEX MIP Community is limited to 1,000 variables or constraints; its $N = 40$ model already has 1,600 binary variables. CP Optimizer Community applies a search-space limit of 2^{1000} , which prevents the recorded $N = 150$ and $N = 200$ models from being evaluated under that license. These limits differ from the academic Gurobi license used for the archived Gurobi rows.

4.2. SAT encoding comparison

Table 2 gives the generated formula sizes at three board dimensions.

Figure 1 plots sizes present in the archived benchmark. At $N = 100$, the pairwise model has 1,646,900 clauses without auxiliary variables, while Sequential uses 49,402 variables. Product produces 117,736 clauses, but its $N = 100$ size is computed from the encoder rather than observed in the archived timing data. It is therefore absent from Figure 1.

Table 3 reports archived construction-plus-solver-call time; dashes denote unmeasured cases. Figure 2 shows construction time, and Figure 3 shows solver invocation time.

Commander has the lowest archived SAT time at $N = 20$ and $N = 50$; Binary is lowest at $N = 75$ and $N = 100$ among the encodings measured at each size. At $N = 150$ and $N = 200$, only Binary has an archived SAT measurement. The archive cannot rank the five encodings at those sizes or explain Sequential's slower $N = 100$ result.

4.3. SAT, ILP, and CP comparison

Table 4 places the ILP and CP baselines beside the minimum recorded SAT time at each size. Its "Best SAT" column is a retrospective minimum, not a prospective solver-selection rule.

Table 4 and Figures 4 and 5 compare the archived methods through $N = 200$. At $N = 200$, Gurobi records 0.601 s, Binary/Glucose4 3.207 s, and CP-SAT 22.487 s for construction plus solver invocation. Binary is the only

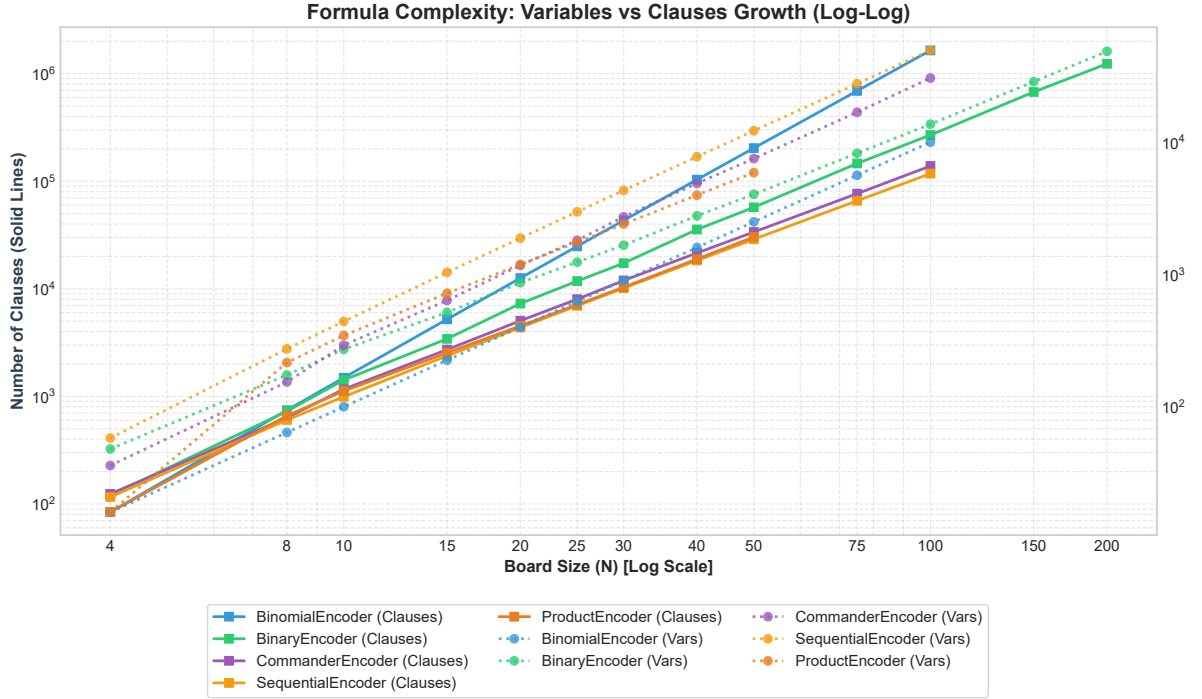


Figure 1: Archived CNF sizes by SAT encoding. Solid lines give clause counts on the left axis, and dotted lines give total variables on the right axis; both axes are logarithmic.

Table 3

Archived construction plus solver invocation time (s); solution extraction excluded

N	Binomial	Binary	Commander	Sequential	Product
8	0.0018	0.0022	0.0021	0.0018	0.0025
20	0.0235	0.3306	0.0192	0.0324	0.6719
50	0.3625	0.1363	0.1212	3.1294	34.7981
75	1.2642	0.3998	0.8540	6.4575	—
100	2.8812	0.5815	0.8140	68.1819	—
150	—	1.5867	—	—	—
200	—	3.2073	—	—	—

SAT encoding measured at this size, and CPLEX Community license limits remove several baseline cells. These observations do not identify a general solver ranking or the cause of any runtime difference.

Figure 6 sorts each method's recorded construction-plus-solver-call times and plots the time of the corresponding case. Its horizontal extent reflects the number of archived measurements, not a common set of attempted cases.

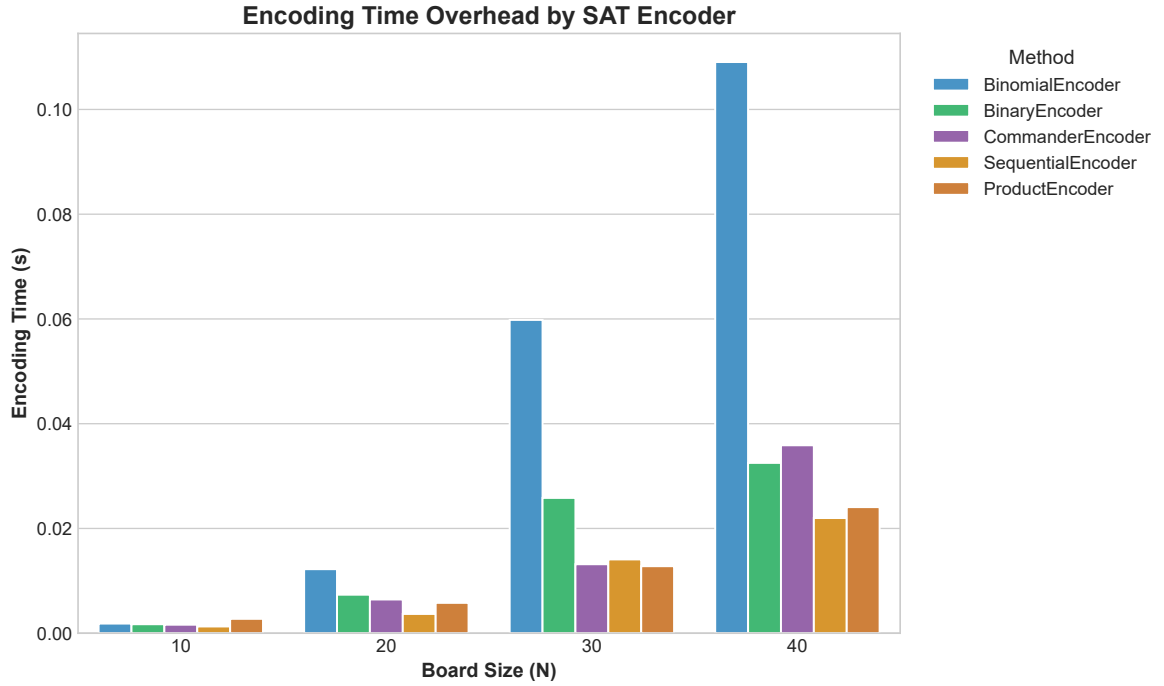


Figure 2: Archived SAT formula-construction time for $N \in \{10, 20, 30, 40\}$; linear time scale.

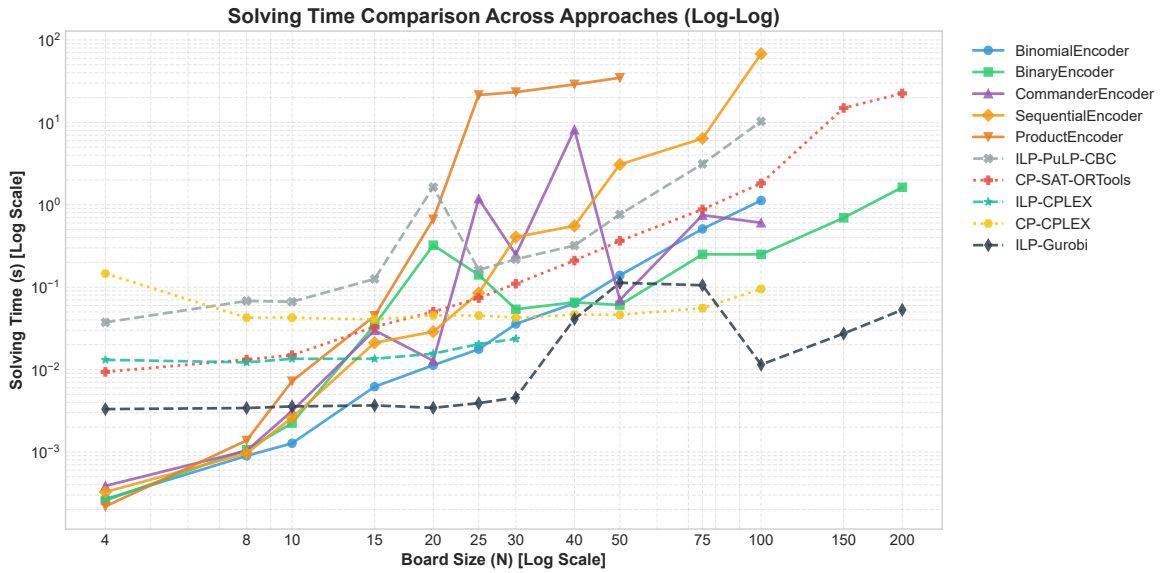
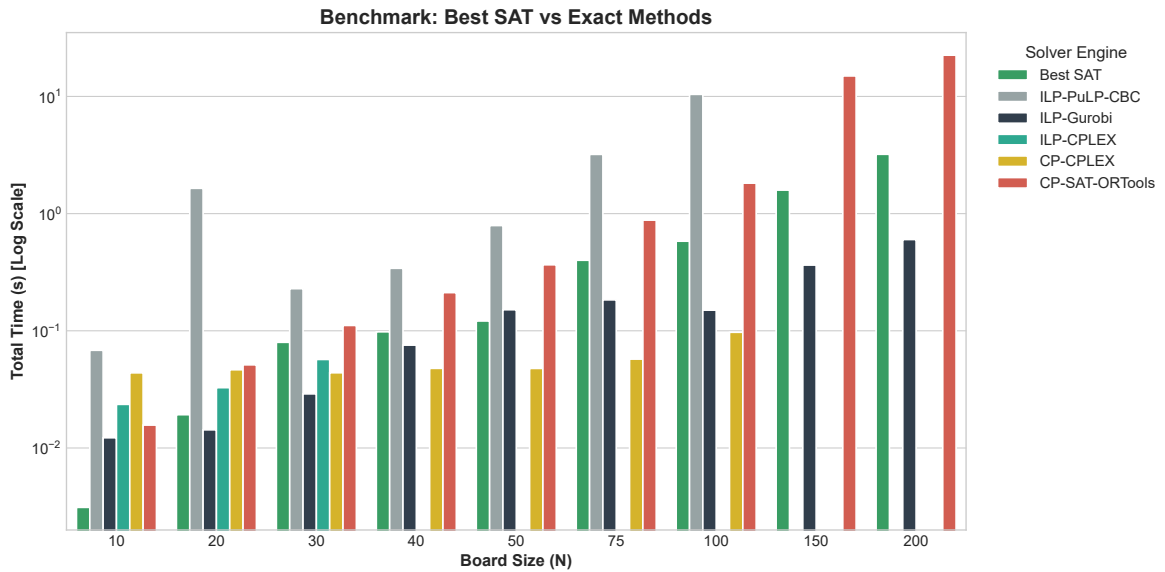


Figure 3: Archived solver invocation time for SAT encodings and ILP/CP baselines on logarithmic axes. Missing method–size combinations were not measured.

Table 4

Archived construction plus solver invocation time (s): retrospective best SAT and baselines

N	Best SAT	ILP (CBC)	ILP (Gurobi)	ILP (CPLEX)	CP (CPLEX)	CP-SAT	Best Method
10	0.003	0.068	0.012	0.024	0.044	0.016	Best SAT
20	0.019	1.644	0.014	0.033	0.046	0.051	Gurobi
30	0.080	0.229	0.029	0.057	0.044	0.111	Gurobi
40	0.098	0.341	0.075	— [†]	0.048	0.212	CPLEX CP
50	0.121	0.791	0.151	— [†]	0.048	0.366	CPLEX CP
75	0.400	3.202	0.184	— [†]	0.057	0.879	CPLEX CP
100	0.581	10.413	0.150	— [†]	0.097	1.819	CPLEX CP
150	1.587	—	0.364	— [†]	— [†]	14.953	Gurobi
200	3.207	—	0.601	— [†]	— [†]	22.487	Gurobi

[†] CPLEX Community license limit; other dashes mean not evaluated.**Figure 4:** Archived construction plus solver invocation time for the retrospective best SAT encoding at each plotted size and the ILP/CP baselines, $N \in \{10, 20, 30, 40, 50, 75, 100, 150, 200\}$. The best-SAT bars may represent different encodings at different sizes; the vertical axis is logarithmic.

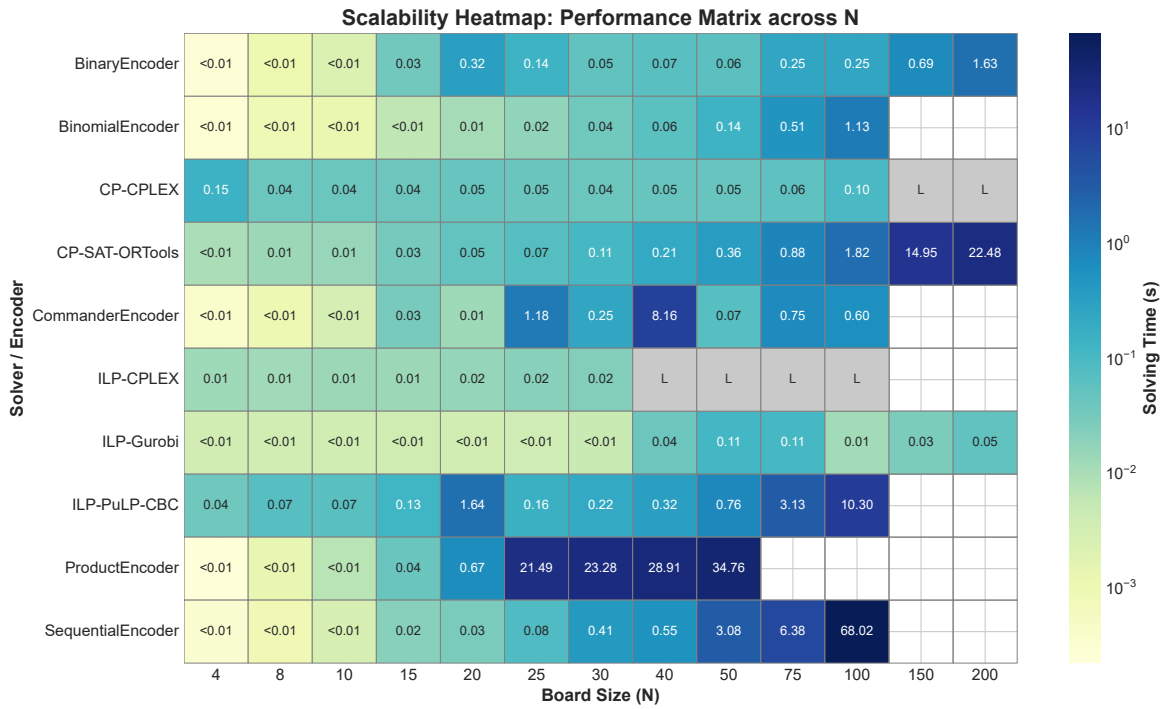


Figure 5: Archived solver invocation time in seconds on a logarithmic color scale. Blank cells were not measured; grey cells marked L denote archived CPLEX Community license failures. Values below 0.01 s are labeled < 0.01.

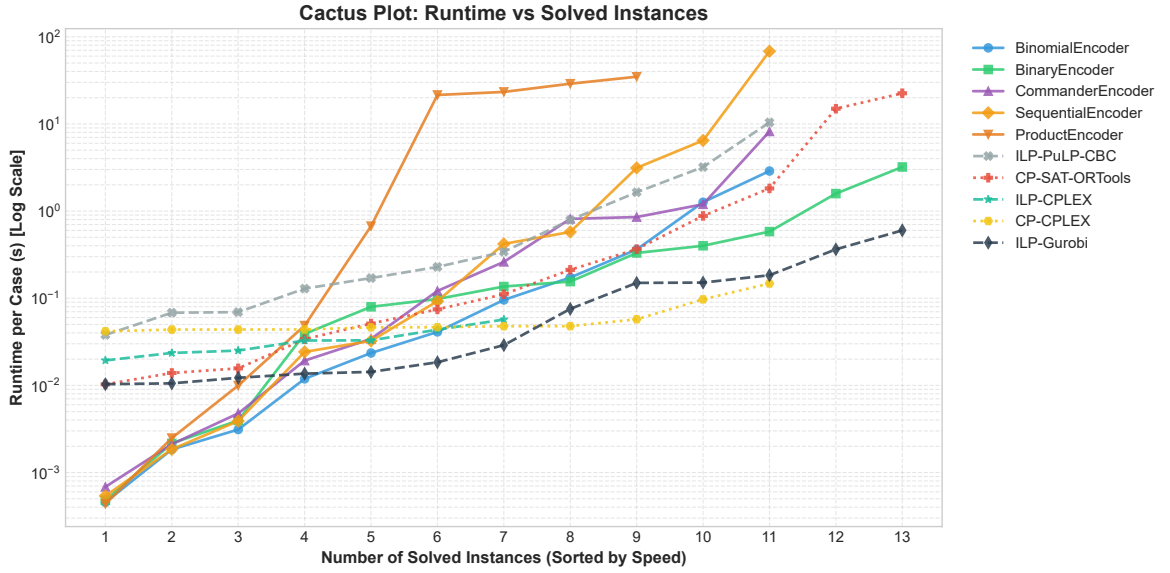


Figure 6: Cactus profile of successful archived construction-plus-solver-call times, sorted separately for each method. The ordinate is per-case time, not cumulative time; a shorter curve reflects fewer recorded cases, not a timeout.

4.4. Controlled protocol check

An earlier controlled run tested Binary/Glucose4, Gurobi, and CP-SAT at five sizes, with three repetitions and a 120-s full-process deadline per case. All 45 runs returned valid placements. The run used one solver worker on Windows 11; its metadata record the processor and package versions. Because the source tree was modified during collection, this run is not tied to a clean commit. Table 5 reports medians. Raw records and metadata are in `new_benchmark/` under `nqueens_sat/results/benchmarks/`.

Table 5: Separate controlled run: median construction plus solver invocation time (s), three repetitions per cell

N	Binary/Glucose4	Gurobi	CP-SAT
20	0.296	0.034	0.047
50	0.140	0.139	0.325
100	0.509	0.120	2.013
150	1.469	0.276	13.269
200	3.163	0.496	49.921

At $N = 200$, the controlled CP-SAT median is 49.921 s, compared with 22.487 s in the archived benchmark. The archived protocol lacks enough detail to explain the difference, so the datasets are not pooled. This controlled run includes only one SAT encoding and cannot rank the other four.

4.5. Clean-source comparison at common sizes

Two clean-source runs tested the same eight methods at $N \in \{20, 50, 100, 150, 200\}$. Each method-size cell had three repetitions, one solver worker, and a 120-s full-process deadline. The Python source was identical in both runs, and every returned placement was validated. The metadata include matching SHA-256 source fingerprints and package versions. Raw records and summaries are in `common_small_benchmark/` and `large_n_benchmark/` under `nqueens_sat/results/benchmarks/`. CPLEX was excluded because its available Community licenses do not cover the full size range.

Table 6 gives medians for the three smaller sizes. Every SAT encoding completed all repetitions. CBC timed out in all three $N = 20$ runs but completed its $N = 50$ and $N = 100$ runs. This non-monotone observation does not imply that the $N = 20$ model is infeasible. The raw CSV records each status and duration.

Table 6: Clean-source smaller-board run: median construction plus solver invocation time (s), three repetitions per cell

Method	$N = 20$	$N = 50$	$N = 100$
Binomial/Glucose4	0.025	0.367	3.237
Binary/Glucose4	0.300	0.131	0.534
Commander/Glucose4	0.019	0.133	0.708
Sequential/Glucose4	0.036	3.000	71.145
Product/Glucose4	0.581	28.036	36.796
CBC	Timeout (3/3)	0.771	9.805
Gurobi	0.035	0.124	0.132
CP-SAT	0.044	0.311	1.740

Table 7 reports medians and observed minimum and maximum times at $N = 150$ and $N = 200$. It shows a median only when all three runs completed. The ranges are descriptive, not confidence intervals; timeout cells have no completion-time median.

Table 7: Clean-source large-board run: median [minimum, maximum] time (s) over three repetitions per cell

Method	$N = 150$	$N = 200$
Binomial/Glucose4	10.397 [10.352, 10.798]	24.805 [24.300, 25.338]
Binary/Glucose4	1.519 [1.391, 1.540]	3.208 [3.054, 3.220]
Commander/Glucose4	1.253 [1.251, 1.288]	2.252 [2.222, 2.358]
Sequential/Glucose4	108.028 [103.949, 108.953]	Timeout (3/3)
Product/Glucose4	1.162 [1.112, 1.205]	2.806 [2.262, 2.860]
CBC	62.080 [60.012, 62.214]	Timeout (3/3)
Gurobi	0.294 [0.260, 0.754]	0.421 [0.420, 0.475]
CP-SAT	12.252 [11.933, 12.491]	44.776 [38.479, 45.935]

Product has the lowest SAT median at $N = 150$; Commander has the lowest at $N = 200$. Gurobi has the lowest median among all tested methods at both sizes. These SAT results differ from the archived “Best SAT” column because only Binary was measured there. Sequential and CBC timed out in all three $N = 200$ runs, so neither has a completion-time median or a SAT/UNSAT result at that size. The reported metric excludes solution extraction and worker startup. Five feasible board sizes and three repetitions per cell cannot establish asymptotic behavior or performance on completion instances.

4.6. Propagation and interpretation

The clauses in Section 3 show that each implemented encoding is GAC on the original inputs of an isolated AMO constraint. This property does not mean that unit propagation alone solves the complete N-Queens formula.

Encoding structure can affect Glucose4’s decisions, propagation, conflicts, and learned clauses [1, 12]. The archived benchmark has no solver-internal counters. The clean-source CSVs record aggregate conflicts, decisions, propagations, and restarts, but not learned-clause LBD distributions or decisions on auxiliary variables. A long implication chain need not span many decision levels, and binary original clauses do not guarantee short learned clauses. The timing data therefore cannot attribute the observed ordering to a specific CDCL mechanism.

Archived figures have unequal method coverage and no run-level uncertainty. The clean-source runs provide raw repetitions at five sizes, but they use one machine and no seeded ablation. Empty-board N-Queens has polynomial constructions, so these timings cannot be generalized to harder completion instances or other combinatorial domains [6].

5. Conclusion and future work

This study compared five established AMO encodings with ILP and CP baselines for empty-board N-Queens. The clean-source runs provide a common-size comparison missing from the archived benchmark. Commander has the lowest SAT median at $N = 200$, while Gurobi has the lowest median among the tested methods. Sequential and CBC timed out in all three runs at that size. These findings concern construction-plus-solver-call time on one machine and do not establish general scalability or performance on hard N-Queens variants.

To test whether encoding structure explains the observed SAT ordering, a follow-up should vary Commander implications, Product grid choices, and symmetry breaking under the same protocol. Learned-clause LBD and branching measurements would help distinguish propagation cost from search effects. To test generality, the same comparison should include feasible and infeasible completion instances, additional seeds, and independent machines.

CRedit authorship contribution statement

Vu Minh Hoang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies

The author used generative AI tools to assist with code revisions, figure design, and LaTeX formatting. The author reviewed the resulting code, figures, and text and remains responsible for the manuscript.

Data availability

The source code, figures, and manuscript are available at <https://github.com/Ohhalo02/nqueens-sat-optimization-01>. The archived aggregate data are in `nqueens_sat/results/benchmark_results.csv`. The three later datasets are in separate directories under `nqueens_sat/results/benchmarks/`; each has per-run records, a summary CSV, and environment metadata. The two clean-source metadata files include the same SHA-256 fingerprint for the Python source used in those runs. The archived data lack raw repetitions and complete provenance, so the four datasets are not pooled.

References

- [1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI-09), pp. 399–404.
- [2] Audemard, G., Simon, L., 2018. Glucose in the SAT 2018 competition, in: Proceedings of SAT Competition 2018: Solver and Benchmark Descriptions, pp. 18–19. URL: <https://helda.helsinki.fi/handle/10138/237063>.
- [3] Bell, J., Stevens, B., 2009. A survey of known results and research areas for n-queens. Discrete Mathematics 309, 1–31. URL: <https://doi.org/10.1016/j.disc.2007.12.043>.
- [4] Chen, J., 2010. A new SAT encoding of the at-most-one constraint, in: Proceedings of the Tenth International Workshop on Constraint Modelling and Reformulation (ModRef 2010), pp. 1–9. URL: <https://archive.modref.org/files/papers/2010/ModRef2010-11-A-New-SAT-Encoding-of-the-At-Most-One-Constraint.pdf>.
- [5] Frisch, A.M., Giannaros, P.A., 2010. SAT encodings of the at-most- k constraint: Some old, some new, some fast, some slow, in: Proceedings of the Tenth International Workshop on Constraint Modelling and Reformulation (ModRef 2010), pp. 1–15. URL: <https://archive.modref.org/files/papers/2010/ModRef2010-13-SAT-Encodings-of-the-At-Most-k-Constraint.pdf>.

- [6] Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n -Queens Completion. *Journal of Artificial Intelligence Research* 59, 815–848. URL: <https://doi.org/10.1613/jair.5512>.
- [7] Google Developers, 2019. Or-tools. URL: <https://developers.google.com/optimization>.
- [8] Gurobi Optimization, LLC, 2025. Gurobi optimizer reference manual. URL: <https://www.gurobi.com>.
- [9] IBM Corporation, 2022. Ibm ilog cplex optimization studio cplex user's manual. URL: <https://www.ibm.com/analytics/cplex-optimizer>.
- [10] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: *International Conference on Theory and Applications of Satisfiability Testing*, Springer. pp. 428–437. URL: https://doi.org/10.1007/978-3-319-94144-8_26.
- [11] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects. *Fourth Workshop on Constraints in Formal Verification (CFV 2007)*. URL: https://www.cs.cmu.edu/~wklieber/papers/2007_efficient-cnf-encoding-for-selecting-1.pdf.
- [12] Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning SAT solvers, in: Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), *Handbook of Satisfiability*. IOS Press. volume 185, pp. 131–153. URL: <https://doi.org/10.3233/978-1-58603-929-5-131>.
- [13] Marques-Silva, J.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers* 48, 506–521. URL: <https://doi.org/10.1109/12.769433>.
- [14] Mitchell, S., O'Sullivan, M., Dunning, I., 2011. PuLP: A Linear Programming Toolkit for Python. Technical Report. University of Auckland. URL: <https://optimization-online.org/wp-content/uploads/2011/09/3178.pdf>.
- [15] Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S., 2001. Chaff: Engineering an efficient SAT solver, in: *Proceedings of the 38th Annual Design Automation Conference*, pp. 530–535. URL: <https://doi.org/10.1145/378239.379017>.
- [16] Nemhauser, G.L., Wolsey, L.A., 1988. *Integer and Combinatorial Optimization*. John Wiley & Sons. URL: <https://doi.org/10.1002/9781118627372>.
- [17] Prestwich, S.D., 2009. CNF encodings, in: *Handbook of Satisfiability*. IOS Press. volume 185, pp. 75–97. URL: <https://doi.org/10.3233/978-1-58603-929-5-75>.
- [18] Régim, J.C., 1994. A filtering algorithm for constraints of difference in cps, in: *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94)*, pp. 362–367.
- [19] Rossi, F., van Beek, P., Walsh, T. (Eds.), 2006. *Handbook of Constraint Programming*. Elsevier. URL: <https://shop.elsevier.com/books/handbook-of-constraint-programming/rossi/978-0-444-52726-4>.
- [20] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: *International Conference on Principles and Practice of Constraint Programming*, Springer. pp. 827–831. URL: https://doi.org/10.1007/11564751_73.